

Superpermutations of length $n! + (n-1)! + (n-2)! + \left(\frac{43}{80} + o(1)\right) (n-3)!$

Jay Pantone
Brief Summary, September 2026
Updated October 2026

1 Results

A *superpermutation* on n symbols is a word that contains every permutation of the n symbols as a contiguous substring. Let $S(n)$ be the shortest possible length, and write

$$F_3(n) = n! + (n-1)! + (n-2)!.$$

The lower bound $S(n) \geq F_3(n) + n - 3$ is due to an anonymous 4chan poster, and was written up by Houston, Pantone, and Vatter [1]. In the other direction, Egan's construction [3], which builds on work of Williams, shows that $S(n) \leq F_3(n) + (n-3)! + n - 3$, and Raudvere [8] has improved this by one for every $n \geq 8$, with a Lean proof. Thus the best general upper bound has had coefficient 1 on $(n-3)!$. Our main result lowers it.

Theorem 1. *For every $\varepsilon > 0$ there is an N such that*

$$S(n) \leq F_3(n) + \left(\frac{43}{80} + \varepsilon\right) (n-3)! \quad \text{for all } n \geq N.$$

Theorem 1 is a consequence of the following explicit bound, which is described in Section 5.

Theorem 2. *Let $n \geq 11$ and $0 \leq \ell \leq n-4$, and put $c = \frac{17}{240}(n-4)!$. Then*

$$S(n) \leq F_3(n) + \frac{43}{80}(n-3)! + (n-4-\ell)c + \ell \min\left(c, \frac{(n-1)!}{(n-1-\ell)!}\right).$$

For example, $n = 11$ and $\ell = 2$ give $S(11) \leq 43,932,117$. Both theorems are proved in Lean (Theorem 1 for rational ε , which is equivalent). The formalization and our words are at <https://github.com/jaypantone/superperm-upper-43-80>. A slight improvement, with coefficient $\frac{4061}{7560} \approx 0.53717$ in place of $\frac{43}{80} = 0.5375$, is described in Section 6 (Theorem 3) but not formalized.

Table 1 gives the lengths of our words and of the upper bounds, and Table 2 compares all of these bounds with the other known bounds. At $n = 8$ and $n = 9$ our words have lengths 46,181 and 408,732. The shortest previously published words we know of at these sizes have lengths 46,204 and 408,965, which is Egan's bound minus one [8]. On September 29, 2026, a word of length 46,188 at $n = 8$ was posted [9].

For lower bounds, Hunter and Raudvere [5] improved the 4chan bound to

$$S(n) \geq F_3(n) + n - 3 + \left\lceil \frac{(n-2)! - (n-2)}{n^2 - 3n + 1} \right\rceil,$$

and Liu [7] improved it further. A forthcoming lower bound of ours, also verified in Lean, is

$$S(n) \geq F_3(n) + n - 3 + \left\lceil \frac{2((n-2)! - (n-1))}{n^2 - 4n + 1} \right\rceil \quad (n \geq 7)$$

in its simplest form; a refinement, also verified in Lean, gives the values in Table 2. All of these lower bounds exceed $F_3(n)$ by only $O((n-4)!)$, so what is now known about the coefficient of $(n-3)!$ is

$$0 \leq \liminf_{n \rightarrow \infty} \frac{S(n) - F_3(n)}{(n-3)!} \leq \limsup_{n \rightarrow \infty} \frac{S(n) - F_3(n)}{(n-3)!} \leq \frac{43}{80}.$$

n	Egan – 1	ours (Lean)	our words	interval	gap
7	5,907	–	–	[5,898, 5,905]	7
8	46,204	46,181	46,181	[46,130, 46,181]	51
9	408,965	408,743	408,732	[408,468, 408,732]	264
10	4,037,046	4,035,009	4,034,889	[4,033,374, 4,034,889]	1,515
11	43,948,807	43,932,117	43,930,680	[43,917,903, 43,930,680]	12,777
12	522,910,088	522,759,498	522,745,581	[522,622,378, 522,745,581]	123,203
13	6,749,568,009	6,748,047,864	6,747,918,066	[6,746,626,519, 6,747,918,066]	1,291,547

Table 1: Upper bounds on $S(n)$. “Ours (Lean)” is the bound proved in Lean. “Our words” gives the lengths of explicit words: the words the construction produces, shortened for $n \geq 9$ by a computer search over where each closed trail is cut open and the order in which the pieces are joined (Section 7); at $n = 12$ and 13 they use the construction of Section 6. “Interval” runs from our lower bound to our shortest word, so it contains $S(n)$, and “gap” is its width. We have no new bounds at $n = 7$; the interval there is due to Lebar [6] and Teodorescu [10].

n	$(n-3)!$	Lower bounds				Upper bounds		
		4chan	Hunter–Raudvere	Liu	ours	our words	ours (Lean)	Egan – 1
8	120	5	23	38	50	101	101	124
9	720	6	98	178	228	492	503	725
10	5,040	7	575	1,080	1,374	2,889	3,009	5,046
11	40,320	8	4,086	7,755	9,423	22,200	23,637	40,327
12	362,880	9	33,301	63,564	75,178	198,381	212,298	362,888
13	3,628,800	10	304,719	584,019	687,319	1,978,866	2,108,664	3,628,809

Table 2: Bounds on $S(n) - F_3(n)$. The lower bound “ours” is our forthcoming Lean-verified bound. “Our words” gives the lengths of our words in Table 1; they are checked by computer but not in Lean. “Ours (Lean)” is the bound proved in Lean: explicit constructions for $n \leq 10$, and Theorem 2 with the best ℓ for $n \geq 11$. “Egan – 1” is Raudvere’s construction [8]. For small n there are sharper special-purpose lower bounds: $S(6) = 872$ [4], $S(7) \geq 5,898$ [6], and $S(8) \geq 46,130$ [2].

2 Slices of 2-loops

Fix an alphabet of $k \geq 4$ symbols. Stepping from a permutation u to a permutation v costs the number of letters needed to extend a word ending in u to one ending in v , namely k minus the

length of the longest suffix of u that is a prefix of v . Write rot for left rotation and $[p]$ for the *cyclic class* of a permutation p , the set of its k rotations (often called a 1-cycle). Stepping to the next rotation costs one.

Fix a *distinguished letter* s and an ordering x of the other $k - 1$ letters. The 2-loop of (x, s) passes through the $k - 1$ cyclic classes $[\text{rot}^j(x)s]$ in order. It goes through each class by steps of cost one, from $\text{rot}^j(x)s$ to $s \text{rot}^j(x)$, and then moves to the next class by the step $a_1 a_2 a_3 \cdots a_k \mapsto a_3 \cdots a_k a_2 a_1$ of cost two, which sends $s \text{rot}^j(x)$ to $\text{rot}^{j+1}(x)s$. Replacing x by a rotation gives the same 2-loop with a different first class, so a 2-loop is determined by s and the cyclic order of x . For fixed s , the $(k - 2)!$ 2-loops partition the permutations.

For example, with $k = 5$, $s = 5$ and $x = 1234$, the 2-loop reads each line below from left to right:

$$\begin{array}{l} 12345 \rightarrow 23451 \rightarrow 34512 \rightarrow 45123 \rightarrow 51234 \\ 23415 \rightarrow 34152 \rightarrow 41523 \rightarrow 15234 \rightarrow 52341 \\ 34125 \rightarrow 41253 \rightarrow 12534 \rightarrow 25341 \rightarrow 53412 \\ 41235 \rightarrow 12354 \rightarrow 23541 \rightarrow 35412 \rightarrow 54123 \end{array}$$

Each line is one cyclic class, and each arrow is a step of cost one. The step from the end of each line to the start of the next costs two, and so does the step from 54123 back to 12345.

A *slice* $\mathcal{S}(x, s; L)$ starts at xs and follows the 2-loop through its first L cyclic classes, so its word has length $(2k - 1) + (L - 1)(k + 1) = (k + 1)L + k - 2$. A slice is *full* if $L = k - 1$ (the whole 2-loop) and *short* if $L = k - 3$ (all but two classes). For example, with $k = 5$, the full slice $\mathcal{S}(1234, 5; 4)$ spells 123451234152341253412354123, and the short slice $\mathcal{S}(1234, 5; 2)$ spells 123451234152341, which contains the classes $[12345]$ and $[23415]$ and misses $[34125]$ and $[41235]$.

The *head* and *tail* of a slice are the first and last $k - 3$ letters of its word. In the example both heads are 12, and the tails are 23 and 41. When the tail of one slice is the head of another, their words can be overlapped on those $k - 3$ letters; the step from the last permutation of the first slice to the first permutation of the second costs three.

A *selection* is a choice of one full or short slice from each 2-loop with distinguished letter s , that is, a rotation of each x together with whether its slice is full or short. Let Q be twice the number of short slices, which is the number of cyclic classes the selection misses. The *endpoint graph* of a selection is the directed multigraph whose vertices are the words of length $k - 3$, which we call *endpoint words*, with one edge from head to tail for each slice. We only use selections whose endpoint graph is balanced, meaning that every vertex has equal in-degree and out-degree. The slices then split into *closed trails*, closed walks in the endpoint graph that use no edge twice. A closed trail of r slices containing A cyclic classes in total spells a word of length $(k + 1)A + r + k - 3$.

3 Adding the last letter

The construction ends with *completion*, which turns a balanced selection on $n - 1$ symbols into closed trails of slices on n symbols that together contain every cyclic class exactly once.

Add a new letter z . For each slice $\mathcal{S}(x, s; L)$ of the selection, write $x = x_0 x_1 \cdots x_{n-3}$, and let $x^{(i)}$ be x with z inserted just before x_i . The slice gives $n - 2$ *inserted slices*, $\mathcal{S}(x^{(i)}, s; L + 1)$ for $i < L$ and

$\mathcal{S}(x^{(i)}, s; L)$ for $i \geq L$. It also gives one *repair slice* for each class $[\text{rot}^i(x)s]$ that it misses: the full slice with distinguished letter z that starts at $\text{rot}^i(x)sz$.

To see that every cyclic class on n symbols is used exactly once, delete z . Each cyclic class on $n - 1$ symbols has $n - 1$ *extensions*, one for each place z can go, and a repair slice consists of all extensions of one missed class. Deleting z from the classes of the 2-loop of $(x^{(i)}, s)$ lists the old classes in order, with $[\text{rot}^i(x)s]$ appearing twice. So the inserted slices, which take one extra class exactly when this repeated class belongs to the old slice, contain exactly the extensions of the old slice's classes. There are $(n - 2)! + Q$ new slices in all.

For example, completing $\mathcal{S}(1234, 5; 2)$ with $z = 6$ gives these six slices:

	First permutation	Cyclic classes	Head	Tail
Inserted	612345	3	612	461
Inserted	162345	3	162	416
Inserted	126345	2	126	341
Inserted	123645	2	123	641
Repair	341256	5	341	412
Repair	412356	5	412	123

They contain all 20 extensions of the four classes of the old 2-loop. In the endpoint graph they form three paths,

$$612 \rightarrow 461, \quad 162 \rightarrow 416, \quad 126 \rightarrow 341 \rightarrow 412 \rightarrow 123 \rightarrow 641,$$

each from one of the three ways to insert 6 into the head 12 to one of the three ways to insert it into the tail 41. Numbering positions by the number of letters in front of 6, the position goes $0 \rightarrow 1$, $1 \rightarrow 2$ and $2 \rightarrow 0$.

This is the general picture. The new slices coming from one old slice form $n - 3$ paths, each from an insertion of z into the head to an insertion of z into the tail. With positions numbered $0, \dots, n - 4$, a short slice sends position j to $j + 1$ and a full slice sends j to $j - 1$, modulo $n - 3$. Since the tail of each old slice is the head of the next, these paths chain together along an old closed trail. Going once around a closed trail with f full and t short slices changes the position by $t - f$, so the trail completes to $\gcd(n - 3, f - t)$ closed trails.

If the completion is a single closed trail, then it contains all $A = (n - 1)!$ cyclic classes in $r = (n - 2)! + Q$ slices, and it spells a word of length

$$(n + 1)(n - 1)! + (n - 2)! + Q + n - 3 = F_3(n) + n - 3 + Q,$$

which is the 4chan lower bound plus Q . This happens at $n = 8$. A computer search found a selection on 7 symbols whose 120 slices form one closed trail, with 72 full and 48 short slices. Since $72 - 48 = 24$ is prime to 5, completion gives one closed trail of 816 slices with $Q = 96$, and a word of length $F_3(8) + 5 + 96 = 46,181$. (Its 96 repeated permutations are exactly where the word enters and leaves each of the 48 two-slice chains of repair slices.)

For large n the difficulty is to keep both Q and the number of closed trails small. Taking every slice full, with each x chosen to end in the same letter, gives a balanced selection with $Q = 0$, but its completion falls into $(n - 3)!$ separate closed trails. The construction below pays for merging trails in two ways, with short slices and with connector cycles, at costs of $\frac{7}{15}(n - 3)!$ and $\frac{17}{240}(n - 3)!$ letters; their sum is $\frac{43}{80}(n - 3)!$.

4 All sizes

Completion needs a balanced selection on $n - 1$ symbols, and we cannot search for a good one separately for every n . Instead we compute one selection and add letters to it one at a time. Completion itself cannot do this, because what it produces is not a selection: its repair slices have distinguished letter z instead of s , and its inserted slices with $i \geq L$ are neither full nor short. We use a different operation, *transport*, which adds a letter and produces a selection again. We transport the computed selection up to $n - 1$ symbols and complete only once, at the end.

Transport from k to $k + 1$ symbols adds a letter w . Each slice $\mathcal{S}(x, s; L)$ gives $k - 1$ slices of the same kind to the new selection, one in each 2-loop whose cyclic order becomes $[x]$ when w is deleted. For a full slice these are $\mathcal{S}(y, s; k)$ for the $k - 1$ words y obtained by inserting w just before a letter of x . For a short slice, write $x = uab$ with a and b letters. The new slices are $\mathcal{S}(y, s; k - 2)$ for the same words y , except that for $uawb$ we use its rotation $buaw$, so that its slice starts at bu , where the slice of $uwab$ ends. This changes which classes are missed, but that does not matter: only the final completion needs every class, and it repairs whatever the last selection misses. For example, transporting $\mathcal{S}(1234, 5; 2)$ with $w = 6$ gives short slices starting at 612345, 162345, 126345 and 412365, which form the paths $612 \rightarrow 461$, $162 \rightarrow 416$ and $126 \rightarrow 412 \rightarrow 641$, with the same ends as in the completion example.

The position rule is the same as for completion, so a closed trail with f full and t short slices becomes $\gcd(k - 2, f - t)$ closed trails. We keep the condition that $k - 2$ divides $f - t$ for every closed trail. Then each closed trail becomes $k - 2$ closed trails, each going once around the old trail, and each with

$$f' - t' = \frac{k - 1}{k - 2}(f - t),$$

because the paths that wrap around between positions 0 and $k - 3$ contain two slices. This is divisible by $k - 1$, so the condition persists. Each transport multiplies Q by $k - 1$ and the number of closed trails by $k - 2$.

Our starting point is a computer-found selection on 10 symbols, with $8! = 40,320$ slices, $Q = 18,816 = \frac{7}{15} \cdot 8!$, and 350 closed trails, each with 8 dividing $f - t$. Transporting it to $n - 1$ symbols and completing gives $Q = \frac{7}{15}(n - 3)!$, in $2800 \cdot (n - 3)!/8!$ closed trails on n symbols.

To join these trails we add *connector cycles*. After completion the endpoint words have length $h = n - 3$. The connector cycle of an endpoint word v is $v \rightarrow \text{rot}(v) \rightarrow \cdots \rightarrow \text{rot}^h(v) = v$. Each step appends one letter, so the cycle costs h letters, and it can be spliced into any closed trail that it meets at a vertex. At $n = 11$, a computer search found 357 connector cycles that together meet all 2800 closed trails. These transport too: from n to $n + 1$, each connector cycle becomes h cycles of length $h + 1$, obtained by inserting w into each of its h gaps, and these meet every trail descended from the trails the old cycle met. For the 231 cycles that contain z , the reason is that the $h + 1$ insertions of w into an endpoint word containing z lie on the $h + 1$ descendants of its trail, one on each. The other 126 cycles contain no z ; each meets the completion of an all-full trail and transports along with it. Hence $c = \frac{17}{240}(n - 4)!$ connector cycles suffice, they cost $hc = \frac{17}{240}(n - 3)!$ letters, and

$$Q + hc = \left(\frac{7}{15} + \frac{17}{240} \right) (n - 3)! = \frac{43}{80}(n - 3)!.$$

5 Joining the pieces

Put all completed slices and connector cycles into one directed multigraph. It is balanced, so each connected component has an Eulerian circuit, and there are $t \leq c$ components because each contains a connector cycle. Choose one connector cycle in each component. Deleting one of its h edges from the component's circuit, which we call a *cut*, leaves a trail that still uses every slice of the component, so it spells a word containing every permutation assigned to that component. A slice with L cyclic classes appends $(n+1)L + 1$ letters after its head, so the slices contribute $(n+1)(n-1)! + (n-2)! + Q = F_3(n) + Q$ letters, and the connector cycles contribute hc . Each word also starts with an endpoint word of h letters, and the cut saves one letter, so the t component words have total length $F_3(n) + Q + hc + (h-1)t$.

Finally, consecutive component words are overlapped. Fix $\ell \leq n-4$, and view each of the h cuts of a component as an edge from the first ℓ letters of the resulting word to its last ℓ letters. As the cut moves around a connector cycle, these prefixes and suffixes run through the same cyclic windows of the cycle's word, so giving each cut the fractional value $1/h$ balances every vertex. Iterative rounding turns this into an actual choice of one cut per component with $|\text{in-degree} - \text{out-degree}| \leq 2$ at every vertex: at each stage, counting ranks shows that some balance equation involves at most three fractional values, and dropping that equation costs at most two. A graph with this property splits into at most as many trails as it has vertices, since in each connected component the positive imbalances add up to at most the number of vertices. Along each trail, consecutive words overlap in ℓ letters. The vertices are words of ℓ distinct letters other than s , so there are at most $(n-1)!/(n-1-\ell)!$ of them. With J trails, the overlaps save $\ell(t-J)$ letters, for a total length of at most

$$F_3(n) + \frac{43}{80}(n-3)! + (n-4-\ell)t + \ell J.$$

Theorem 2 follows from $t \leq c$ and $J \leq \min(t, (n-1)!/(n-1-\ell)!)$. For Theorem 1, take $\ell = n-2-a$, where a is the least integer with $(a+1)! \geq (n-2)^4$. Then $a = O(\log n / \log \log n)$, and the two correction terms are $O(a/n)(n-3)!$ and $O(1/n)(n-3)!$.

6 A slight improvement

Theorem 3. *For every $\varepsilon > 0$ there is an N such that*

$$S(n) \leq F_3(n) + \left(\frac{4061}{7560} + \varepsilon \right) (n-3)! \quad \text{for all } n \geq N.$$

Here $\frac{4061}{7560} = \frac{43}{80} - \frac{1}{3024}$. The construction is unchanged except for the connector cycles. Section 4 chooses them at $n = 11$ and transports them to every larger n , and an integer program over all candidate cycles shows that no fewer than 357 cycles meet the 2800 closed trails at $n = 11$. One level up, however, the transported cycles are not optimal. Transport the selection on 10 symbols once, to 11 symbols, and complete it to 12. Each all-full trail then completes to 9 closed trails that are all met by one connector cycle without z , which accounts for 2688 cycles. The other 1008 closed trails are met by 155 cycles containing z , found by an integer program. This gives 2843 cycles in place of the $357 \cdot 8 = 2856$ transported ones. The argument of Section 4 now starts at $n = 12$, with

$c = \frac{2843}{40320}(n-4)!$, and gives the coefficient $\frac{7}{15} + \frac{2843}{40320} = \frac{21659}{40320}$. At $n = 13$ a local search finds 25,584 cycles, three fewer than the 25,587 obtained by transporting these 2843, and this gives Theorem 3.

The cycles at $n = 12$ and at $n = 13$ are each checked by two independently written programs; neither set is formalized in Lean. Choosing the cycles at still larger n cannot go much further. A connector cycle of length h meets at most h closed trails, so hc is at least the number $\frac{5}{72}(n-3)!$ of closed trails, and the coefficient of this construction is at least $\frac{7}{15} + \frac{5}{72} = \frac{193}{360} \approx 0.53611$.

7 The words, and what is verified

At $n = 8$ our word is the single completed trail of Section 3. At $n = 9$ we complete a selection on 8 symbols with two closed trails, which gives 52 closed trails on 9 symbols that are then joined by overlaps. At $n = 10$ we transport the same selection once and use 54 connector cycles. At $n = 11$ we use the construction of Theorem 2, and at $n = 12$ and 13 that of Section 6. For $9 \leq n \leq 13$ we then optimized the cuts and the order of the component words. Besides the cuts of Section 5, a component may be opened at other positions, such as any of its endpoint words; this costs extra letters but can give longer overlaps. The choices come from integer programming and local search. At $n = 12$ and 13 the components fall into 48 large groups that are relabellings of one another, so the best arrangement found for one group can be copied to all 48. At $n = 10$, the six connector cycles that meet 28 of the closed trails are also replaced by one open path through all 28. This improves on the lengths proved in Lean by between 11 letters (at $n = 9$) and 129,798 letters (at $n = 13$).

AI Disclosure: This project made heavy use of AI models (largely OpenAI’s ChatGPT 6 Astra and Anthropic’s Claude Opus 5.5) for research, formalization, and drafting this summary.

References

- [1] Anonymous 4chan poster, R. Houston, J. Pantone, and V. Vatter, *A lower bound on the length of the shortest superpattern*. <https://oeis.org/A180632/a180632.pdf>.
- [2] W. Echols, *superperm8-ge-46130*, Lean repository. <https://github.com/willamechols/superperm8-ge-46130>.
- [3] G. Egan, *Superpermutations*. <https://www.gregegan.net/SCIENCE/Superpermutations/Superpermutations.html>.
- [4] B. Grayzel et al., *superperm6*, Lean repository. <https://github.com/BGray-wrl/superperm6>.
- [5] Z. Hunter and U. Raudvere, *superpermutations-hunter*, Lean repository. <https://github.com/urdvr/superpermutations-hunter>. Announcement: <https://groups.google.com/g/superpermutators/c/Uo2-KjoDYbg>.
- [6] J. Lebar, *superperm7-ge-5898*, Lean repository. <https://github.com/jlebar/superperm7-ge-5898>.
- [7] X. Liu, *Preimage-chain corrections, primitive-block rigidity, and new lower bounds for superpermutations*, with Lean repository. <https://github.com/Haruhiyuki/superpermutations-preimage-chain-lower-bounds>.

- [8] U. Raudvere, *superpermutation-examples*, Lean repository.
<https://github.com/urdvr/superpermutation-examples>. Announcement:
<https://groups.google.com/g/superpermutators/c/TBZsctlczM4>.
- [9] rumstd, *superpermutation for $n=8$ symbols of length 46188*, pull request to the superperm repository, September 29, 2026. <https://github.com/superpermutators/superperm/pull/56>. Announcement:
<https://groups.google.com/g/superpermutators/c/Iq23Ghr7a-I>.
- [10] A. Teodorescu, seven-symbol superpermutations of length 5,905, posts to the superpermutators group, August 2026. <https://groups.google.com/g/superpermutators/c/D5sKkV2jBuc>.